

Proyecto Arduino Pantalla Poema

<https://gemini.google.com/app/c71b4223e1128105>

User prompt: Estoy trabajando en un proyecto de arduino, para presentar un poema en una pantalla led, específicamente esta: Pantalla LCD Azul 16x02. Tengo un código inicial funcional. A continuación te indico cuales son las cosas que queremos que pasen: 1. Inicia el Arduino 2. El texto versosPoema[] avanzará de manera continua hasta que se presionen los botones o se manipule el potenciómetro 3. En caso de ser solo 1, el texto se congela y no sigue avanzando hasta que se deje de presionar 4. Si son los 2 botones, se detiene el texto y desaparece 5. Mientras esto ocurre, se consulta en qué sección del texto se encuentra 6. Acción a: si los 2 botones se mantienen presionados menos de 1 segundo: En base a la sección del texto mostrado, se visualizará una palabra clave que nosotros elegimos por cada verso: "firme y fuerte", "acercandose", "estallan", "en cada", "tierra", "contentos", "oprimiran", "algun", "silencio", "amado mio", "injusto", "aislan", "lugar", "muerte" 7. Acción b: Si los 2 botones se mantienen presionados por más de 1 segundo: aparece un nuevo poema conformado por las palabras anteriores en una sola línea. 8. Para que esto ocurra sin interferencias considera esperar un tiempo mayor a 1 segundo (ej: 1,2 o 1,5) desde que se presionan ambos botones para definir cual de las dos posibles acciones ocurre según el tiempo accionado. 9. En el caso de que mientras esté ocurriendo la acción b se deje de presionar 1 solo botón, vuelve a versosPoema[] al punto donde quedó el verso congelado. 10. Pero si se sueltan ambos botones, desaparece el poemaNuevo[] 11. Luego continúa avanzando el versoPoema[] desde el punto exacto en el que quedó congelado 12. En caso de ser manipulado el potenciómetro, se modificará la velocidad del texto (mientras más lejos del centro, más rápido el cambio) 12. Si se rota hacia la derecha avanza de manera normal solo variando la velocidad 13. Si se rota hacia la izquierda retrocede el texto, según qué tan lejos del centro esté. Entonces: a. ayúdame a integrar 2 botones, sin alterar otras cosas que no sean necesarias en el código. Y una vez listo indicame como realizar una prueba en thinkercad. b. No borres los comentarios que ya están en el código original, ya que son importantes para nosotros entender que es lo que hemos hecho.

Response: ¡Qué buen proyecto estás armando! Es un nivel de detalle súper interesante para interactuar con la pantalla.

Le agregué la lógica de los dos botones respetando tal cual cómo funcionaba el potenciómetro y manteniendo todos tus comentarios originales para que no te pierdas. Para hacerlo simple y no usar resistencias extra (que a veces complican el cableado), usé la función INPUT_PULLUP de Arduino.

Aquí tienes el código listo para usarse:

```
#include <LiquidCrystal.h>

// versos del poema
const char *versosPoema[] = {
  "Cuando estan nuestras almas frente a frente,",
  "mudas, erguidas, fuertes, ya muy proximas,",
  "y sus alas se encienden al tocarse,",
  "en cada punta curva ?que mal amargo",
  "puede hacernos la tierra, que no debiéramos",
  "quedarnos aqui, contentos? Piensalo. Al subir más alto,",
  "los angeles nos oprimirian y aspirarian",
  "a dejar caer algun aureo orbe de canto perfecto",
  "en nuestro hondo, querido silencio. Quedemonos",
  "mejor en la tierra, Amado mio, donde los animos",
  "contrarios e injustos de los hombres retroceden",
  "y aislan a los espíritus puros, y permiten",
  "un lugar donde estar y amar por un dia,",
  "con la oscuridad y la hora de la muerte rodeandolo."
};

// corresponde a los pines que utiliza la pantalla
// pantalla lcd verde 16 x 02 con controlador SPLC780D1 o HD44780
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// texto que se muestra al iniciar el dispositivo
const char textoInicialCC[] = "[PLACEHOLDER] - CC BY-SA 4.0"; // licencia de uso, Creative Commons BY-SA 4.0
const char textoInicialTitulo[] = "Cuando nuestras dos almas se eleven"; // titulo del poema
const char textoInicialAutora[] = "Elizabeth Barret Brown"; // autora del poema

// --- variables para los botones y nuevas acciones ---
const int boton1Pin = 6;
const int boton2Pin = 7;
int estadoActual = 0; // 0: Normal, 1: Congelado (1 botón), 2: Dos botones presionados
unsigned long tiempoInicioDosBotones = 0;
unsigned long lastScrollNuevoPoema = 0;
int posNuevoPoema = 0;

// palabras clave por cada verso
const char *palabrasClave[] = {
  "firme y fuerte", "acercandose", "estallan", "en cada", "tierra",
  "contentos", "oprimiran", "algun", "silencio", "amado mio",
  "injusto", "aislan", "lugar", "muerte"
};

// nuevo poema conformado por las palabras clave
const char poemaNuevo[] = "firme y fuerte acercandose estallan en cada tierra contentos oprimiran algun silencio amado m
```

```

//----- variables pote -----
// variables y constantes
// para lectura potenciómetro
const int potePatita = A0;
int poteLectura = -1;
int poteFiltrado = -1;

// funcion entera
// para tomar una variable entera original
// y dividirla por otro entero para perder resolucio
int filtrarConDivision(int valor, int divisor) {
    int resultado = valor / divisor;
    return resultado;
}
// ---- fin variables pote ----

// determinar dirección y calcular velocidad
// relacionada al desplazamiento del poema
int direccion = 0; // variable asociada a si el texto avanza o retrocede
int pausa = 0;
int v = 0;
int pos = 0; // variable que determina la posición del texto

// ---- inicio de funcionamiento ----

void setup() {

    Serial.begin(9600);
    lcd.begin(16, 2); //define el tamaño de la pantalla

    // Configuración de pines para los botones usando resistencias internas del Arduino
    pinMode(boton1Pin, INPUT_PULLUP);
    pinMode(boton2Pin, INPUT_PULLUP);

    // --- texto inicial 1: textoInicialCC / Creative Commons BY - SA ---

    lcd.setCursor(0, 0); //define la seccion superior de la pantalla
    for(int i = 0; i < 16 && textoInicialCC[i] != '\0'; i++) {
        lcd.print(textoInicialCC[i]);
    }
    lcd.setCursor(0, 1); //define la seccion inferior de la pantalla
    lcd.print(textoInicialCC + 16);

    delay(4000);
    lcd.clear();

    // --- texto inicial 2: Carrusel de textoInicialB en la fila inferior (0, 0) ---
    int largoB = strlen(textoInicialTitulo); // calculam el largo (35 letras)

    // calcula cuántos pasos debe avanzar para mostrarlo todo
    // si el texto es más corto de 16, no se mueve (0 pasos)
    int pasosTotales = (largoB > 16) ? (largoB - 16 + 3) : 0; // +3 para dejar unos espacios al final

    for(int pos = 0; pos <= pasosTotales; pos++) {
        lcd.setCursor(0, 0);

        // imprime la "ventana" de 16 caracteres
        for(int i = 0; i < 16; i++) {
            if (pos + i < largoB) {
                lcd.print(textoInicialTitulo[pos + i]);
            } else {
                lcd.print(' '); // rellena con espacios en blanco cuando se acaba el texto
            }
        }

        // si esta en el primer cuadro (pos = 0), hace una pausa más larga
        // para que se pueda empezar a leer antes de que se mueva
        if (pos == 0) {
            delay(2000);
        } else {

```

```

    delay(500); // velocidad del carrusel (350ms por letra)
  }
}

lcd.clear();

// --- texto inicial 3: textoInicialC / autora---
lcd.setCursor(0, 0);
for(int i = 0; i < 16 && textoInicialAutora[i] != '\0'; i++) {
  lcd.print(textoInicialAutora[i]);
}
lcd.setCursor(0, 1);
lcd.print(textoInicialAutora + 16);

delay(4000);
lcd.clear();
}

void loop() {

  // Lectura de los botones (LOW significa presionado porque usamos INPUT_PULLUP)
  bool b1 = digitalRead(boton1Pin) == LOW;
  bool b2 = digitalRead(boton2Pin) == LOW;

  // LÓGICA DE LOS BOTONES
  if (b1 && b2) {
    // Si se presionan ambos botones
    if (estadoActual != 2) {
      estadoActual = 2;
      tiempoInicioDosBotones = millis();
      lcd.clear(); // Borra el texto de golpe
      posNuevoPoema = 0; // Prepara el nuevo poema desde el inicio
      lastScrollNuevoPoema = millis();
    }

    unsigned long tiempoPresionado = millis() - tiempoInicioDosBotones;

    if (tiempoPresionado < 1200) { // Acción A: Menos de 1.2 segundos (1200 ms)
      lcd.setCursor(0, 0);
      lcd.print(palabrasClave[v]); // Muestra la palabra clave de la sección actual

      // Rellena el resto de la pantalla con espacios para que no quede basura visual
      for(int i = strlen(palabrasClave[v]); i < 16; i++) {
        lcd.print(' ');
      }
    } else { // Acción B: Más de 1.2 segundos (Nuevo poema largo que avanza)
      unsigned long currentMillis = millis();

      // Controla la velocidad a la que avanza el poema nuevo sin usar delay
      if (currentMillis - lastScrollNuevoPoema > 300) {
        posNuevoPoema++;
        int largoPoemaNuevo = strlen(poemaNuevo);
        if (posNuevoPoema > largoPoemaNuevo - 16) {
          posNuevoPoema = 0; // Si llega al final, vuelve a empezar
        }
        lastScrollNuevoPoema = currentMillis;
      }

      lcd.setCursor(0, 0);
      for (int i = 0; i < 16; i++) {
        if (posNuevoPoema + i < strlen(poemaNuevo)) {
          lcd.print(poemaNuevo[posNuevoPoema + i]);
        } else {
          lcd.print(' ');
        }
      }
    }
    return; // Sale del loop para que no se ejecute el potenciómetro ni avance el verso normal
  }
  else if (b1 || b2) {
    // Si se presiona solo 1 botón (o se soltó uno estando en la Acción B)
    if (estadoActual != 1) {
      estadoActual = 1;
    }
  }
}

```

```

    lcd.clear(); // Limpia para asegurar que se borre el poemaNuevo si veníamos de ahí
}

// Imprime el texto original tal cual estaba, congelado en su posición actual
int largoVerso = strlen(versosPoema[v]);
lcd.setCursor(0, 0);
for (int i = 0; i < 16; i++) {
    if (pos + i < largoVerso && pos + i >= 0) {
        lcd.print(versosPoema[v][pos + i]);
    } else {
        lcd.print(' ');
    }
}
return; // Sale del loop para que no actualice la posición y se quede congelado
}
else {
    // Ningún botón presionado: Funcionamiento normal con el potenciómetro
    if (estadoActual != 0) {
        estadoActual = 0;
        lcd.clear(); // Limpia la pantalla para quitar rastro de las acciones de botones
    }

    // ----- lectura pote -----
    // función para leer el potenciómetro
    // lectura de pin A0
    // conectar pin 2 de pote
    // lectura va de 0 a 1024
    poteLectura = analogRead(potePatita);

    // division de lectura de pote
    // valor resultante va de 0 a 255
    poteFiltrado = filtrarConDivision(poteLectura, 4);

    // imprimir en el monitor serial el poteFiltrado
    Serial.print("valor filtrado ");
    Serial.println(poteFiltrado);
    // ----- fin lectura pote -----

    if (poteFiltrado >= 135) {
        direccion = 1; // Avanzar
        // Mapea desde 135 (el mínimo para avanzar) hasta 255 (velocidad máxima)
        pausa = map(poteFiltrado, 135, 255, 600, 50);
    }
    else if (poteFiltrado <= 120) {
        direccion = -1; // Retroceder
        // Mapea desde 120 (el mínimo para retroceder) hasta 0 (velocidad máxima en reversa)
        pausa = map(poteFiltrado, 120, 0, 600, 50);
    }
    else {
        direccion = 0; // Pausa / Zona muerta al centro (valores entre 121 y 134)
        pausa = 200; // Pequeño delay de espera
    }

    // calcula el verso actual
    int totalVersos = sizeof(versosPoema) / sizeof(versosPoema[0]);
    int largoVerso = strlen(versosPoema[v]);
    int pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;

    // imprimir el texto en la pantalla
    lcd.setCursor(0, 0);
    for (int i = 0; i < 16; i++) {
        if (pos + i < largoVerso && pos + i >= 0) {
            lcd.print(versosPoema[v][pos + i]);
        } else {
            lcd.print(' ');
        }
    }

    // 4. Aplicar la velocidad calculada
    delay(pausa);

    // 5. Actualizar la posición para el siguiente ciclo
    pos += direccion;
}

```

```
// 6. Lógica para cambiar de verso si llegamos al límite (avanzando o retrocediendo)
if (pos > pasosTotales) {
  // Si avanza más allá del verso actual, pasa al siguiente
  pos = 0;
  v++;
  if (v >= totalVersos) v = 0; // Vuelve al inicio si terminó el poema
  lcd.clear();
  delay(100); // Pausa visual al cambiar de línea
}
else if (pos < 0) {
  // Si retrocede más allá del inicio, vuelve al verso anterior
  v--;
  if (v < 0) v = totalVersos - 1; // Va al último verso si retrocede desde el inicio

  // Recalcula el tamaño del nuevo verso para posicionarse al final de este
  largoVerso = strlen(versosPoema[v]);
  pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;
  pos = pasosTotales;

  lcd.clear();
  delay(100);
}
}
```

¿Cómo probarlo en Tinkercad?

Hacer la prueba en el simulador es súper sencillo con este código:

1. **Saca 2 "Pulsadores" (pushbuttons)** del panel de componentes.
2. Cada pulsador tiene patitas a los lados. Conecta **una patita del primer botón al Pin 6** del Arduino.
3. Conecta **una patita del segundo botón al Pin 7** del Arduino.
4. Las patitas contrarias de ambos botones conéctalas directo a **GND (Tierra)**.

No necesitas agregarle resistencias a la protoboard para estos botones. Al escribir `pinMode(boton1Pin, INPUT_PULLUP);` en el código, le estamos diciendo al Arduino que active unas resistencias pequeñitas que ya trae adentro por defecto, dejándonos el circuito de afuera mucho más limpio. Cuando apretes los botones, el sistema leerá perfectamente los comandos para congelar, lanzar la palabra clave (si los tocas rapidito) o lanzar el poema largo (si los mantienes pegados un buen rato).

User prompt: Cómo se conecta exactamente, ya que tiene 4 patitas, el lado plano creo que contiene ambas patas de abajo negativas y ambas superiores positivas, corrígeme si estoy mal.

Response: Hay un pequeño error de concepto ahí: **estos botones no tienen polaridad**, es decir, no tienen un lado positivo ni uno negativo. Funcionan simplemente como un "puente" mecánico que se cierra al presionarlo.

Internamente, aunque veas 4 patitas en la imagen, en realidad funcionan en pares. Las patitas que están en los lados más cercanos entre sí ya están unidas por dentro por un fierrito.

Para que nunca te equivoques al conectarlos ni tengas que adivinar qué lado es cuál, el truco infalible en electrónica es **usar siempre las patitas que están en diagonal opuesta**.

Así es como debes conectarlo en la protoboard para que funcione con el código que armamos (que usa resistencias internas):

1 Ubica el botón en la protoboard

Paso clave

Coloca el botón de manera que quede "montado" justo sobre el canal o surco central de tu protoboard. Esto separa físicamente las patas de la izquierda de las de la derecha.

2 Conecta la señal al Arduino

Pin digital

Elige una patita cualquiera del botón y conéctala directamente al **Pin 6** (o al Pin 7 para tu segundo botón).

3 Conecta a Tierra

GND

Busca la patita que está exactamente en la **diagonal opuesta** a la que acabas de usar, y conecta esa pata a cualquier pin **GND** del Arduino.

Al conectarlo en diagonal, te aseguras al 100% de que estás usando los dos extremos separados del interruptor, sin importar hacia dónde mire la muesca o el lado plano del botón. Las otras dos patitas que sobran simplemente las dejas sin conectar a nada.

¿Tienes la protoboard a mano para probar esta posición o necesitas ayuda viendo cómo cruzar el canal central?

User prompt: Si quiero usar una resistencia en vez del input pullup, ¿de cuánto debe ser la resistencia que use y en qué punto específico debo colocarla?

Response: Lo ideal y lo más estándar en electrónica para este tipo de botones es usar una resistencia de **10kΩ (10,000 ohmios)**. Si tienes una a mano, sus franjas de colores deberían ser Café, Negro, Naranja.

Como el código que armamos detecta cuando el botón se presiona leyendo un nivel bajo (LOW), necesitamos hacer una configuración que se llama **Pull-up externo**. Esto mantiene la señal en alto (5V) cuando el botón está en reposo, y la manda a tierra (GND) cuando lo aprietas.

Así es exactamente como debes conectarlo en la protoboard:

1 Conecta la resistencia a 5V

Voltaje

Mantén el botón en el canal central. Conecta un extremo de la resistencia de 10k a la línea de **5V**, y el otro extremo a una de las patitas del botón.

2 Conecta la señal al Arduino

Pin digital

Desde esa **misma patita** del botón donde acabas de enchufar la resistencia, saca un cable directo hacia el Pin 6 (o el Pin 7 para el segundo botón) del Arduino.

3 Cierra el circuito a Tierra

GND

Busca la patita del botón que está en la **diagonal opuesta** y conéctala directo a la línea **GND** (Tierra).

callout-content-4

Por último, como ahora la resistencia física está haciendo el trabajo pesado, tienes que ir al `setup()` de tu código y cambiar la instrucción de los botones. Tienes que borrar el `_PULLUP` para que quede solamente como `INPUT`.

Quedaría así:

```
pinMode(boton1Pin, INPUT);
pinMode(boton2Pin, INPUT);
```

¿Tienes resistencias de 10k a mano o quieres ver si nos sirve algún otro valor de las que tengas por ahí?

User prompt: Entonces, necesito que me ayudes a cambiar el código para no utilizar las resistencias que el arduino coloca en cada botón, sino que utilizar resistencias de 10k para configurarlas como pulldown en la protoboard. No edites absolutamente nada más del código y mantén los comentarios originales.

Response: ¡Perfecto! Para usar resistencias en configuración **pull-down**, la lógica se invierte: cuando el botón está en reposo la resistencia lo "tira" hacia tierra (leyendo un LOW), y cuando lo presionas, la corriente pasa directo desde los 5V, por lo que el Arduino lee un HIGH.

Para adaptar esto en el código, solo modifiqué dos cosas muy puntuales sin tocar nada más de la estructura ni borrar tus comentarios:

1. Cambié `INPUT_PULLUP` por `INPUT` en el `setup()`.
2. Cambié la lectura en el `loop()` para que detecte el botón presionado cuando está en HIGH en lugar de LOW.

Aquí tienes el código actualizado listo para copiar y pegar:

```
#include <LiquidCrystal.h>

// versos del poema
const char *versosPoema[] = {
  "Cuando estan nuestras almas frente a frente,",
  "mudas, erguidas, fuertes, ya muy proximas,",
  "y sus alas se encienden al tocarse,",
  "en cada punta curva ?que mal amargo",
  "puede hacernos la tierra, que no debiéramos",
  "quedarnos aqui, contentos? Piensalo. Al subir más alto,",
  "los angeles nos oprimirian y aspirarian",
  "a dejar caer algun aureo orbe de canto perfecto",
  "en nuestro hondo, querido silencio. Quedemonos",
  "mejor en la tierra, Amado mio, donde los animos",
  "contrarios e injustos de los hombres retroceden",
  "y aislan a los espíritus puros, y permiten",
```

```

    "un lugar donde estar y amar por un dia,",
    "con la oscuridad y la hora de la muerte rodeandolo."
};

// corresponde a los pines que utiliza la pantalla
// pantalla lcd verde 16 x 02 con controlador SPLC780D1 o HD44780
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// texto que se muestra al iniciar el dispositivo
const char textoInicialCC[] = "[PLACEHOLDER] - CC BY-SA 4.0"; // licencia de uso, Creative Commons BY-SA 4.0
const char textoInicialTitulo[] = "Cuando nuestras dos almas se eleven"; // titulo del poema
const char textoInicialAutora[] = "Elizabeth Barret Brown"; // autora del poema

// --- variables para los botones y nuevas acciones ---
const int boton1Pin = 6;
const int boton2Pin = 7;
int estadoActual = 0; // 0: Normal, 1: Congelado (1 botón), 2: Dos botones presionados
unsigned long tiempoInicioDosBotones = 0;
unsigned long lastScrollNuevoPoema = 0;
int posNuevoPoema = 0;

// palabras clave por cada verso
const char *palabrasClave[] = {
    "firme y fuerte", "acercandose", "estallan", "en cada", "tierra",
    "contentos", "oprimiran", "algun", "silencio", "amado mio",
    "injusto", "aislan", "lugar", "muerte"
};

// nuevo poema conformado por las palabras clave
const char poemaNuevo[] = "firme y fuerte acercandose estallan en cada tierra contentos oprimiran algun silencio amado m

//----- variables pote -----
// variables y constantes
// para lectura potenciómetro
const int potePatita = A0;
int poteLectura = -1;
int poteFiltrado = -1;

// funcion entera
// para tomar una variable entera original
// y dividirla por otro entero para perder resolucion
int filtrarConDivision(int valor, int divisor) {
    int resultado = valor / divisor;
    return resultado;
}
// ---- fin variables pote ----

// determinar dirección y calcular velocidad
// relacionada al desplazamiento del poema
int direccion = 0; // variable asociada a si el texto avanza o retrocede
int pausa = 0;
int v = 0;
int pos = 0; // variable que determina la posición del texto

// ----- inicio de funcionamiento ----

void setup() {
    Serial.begin(9600);
    lcd.begin(16, 2); //define el tamaño de la pantalla

    // Configuración de pines para los botones usando resistencias externas (pull-down)
    pinMode(boton1Pin, INPUT);
    pinMode(boton2Pin, INPUT);

    // --- texto inicial 1: textoInicialCC / Creative Commons BY - SA ---

    lcd.setCursor(0, 0); //define la seccion superior de la pantalla
    for(int i = 0; i < 16 && textoInicialCC[i] != '\0'; i++) {
        lcd.print(textoInicialCC[i]);
    }
}

```

```

lcd.setCursor(0, 1); //define la seccion inferior de la pantalla
lcd.print(textoInicialCC + 16);

delay(4000);
lcd.clear();

// --- texto inicial 2: Carrusel de textoInicialB en la fila inferior (0, 0) ---
int largoB = strlen(textoInicialTitulo); // calculam el largo (35 letras)

// calcula cuántos pasos debe avanzar para mostrarlo todo
// si el texto es más corto de 16, no se mueve (0 pasos)
int pasosTotales = (largoB > 16) ? (largoB - 16 + 3) : 0; // +3 para dejar unos espacios al final

for(int pos = 0; pos <= pasosTotales; pos++) {
    lcd.setCursor(0, 0);

    // imprime la "ventana" de 16 caracteres
    for(int i = 0; i < 16; i++) {
        if (pos + i < largoB) {
            lcd.print(textoInicialTitulo[pos + i]);
        } else {
            lcd.print(' '); // rellena con espacios en blanco cuando se acaba el texto
        }
    }

    // si esta en el primer cuadro (pos = 0), hace una pausa más larga
    // para que se pueda empezar a leer antes de que se mueva
    if (pos == 0) {
        delay(2000);
    } else {
        delay(500); // velocidad del carrusel (350ms por letra)
    }
}

lcd.clear();

// --- texto inicial 3: textoInicialC / autora---
lcd.setCursor(0, 0);
for(int i = 0; i < 16 && textoInicialAutora[i] != '\0'; i++) {
    lcd.print(textoInicialAutora[i]);
}
lcd.setCursor(0, 1);
lcd.print(textoInicialAutora + 16);

delay(4000);
lcd.clear();
}

void loop() {

    // Lectura de los botones (HIGH significa presionado porque usamos configuración pull-down)
    bool b1 = digitalRead(boton1Pin) == HIGH;
    bool b2 = digitalRead(boton2Pin) == HIGH;

    // LÓGICA DE LOS BOTONES
    if (b1 && b2) {
        // Si se presionan ambos botones
        if (estadoActual != 2) {
            estadoActual = 2;
            tiempoInicioDosBotones = millis();
            lcd.clear(); // Borra el texto de golpe
            posNuevoPoema = 0; // Prepara el nuevo poema desde el inicio
            lastScrollNuevoPoema = millis();
        }

        unsigned long tiempoPresionado = millis() - tiempoInicioDosBotones;

        if (tiempoPresionado < 1200) { // Acción A: Menos de 1.2 segundos (1200 ms)
            lcd.setCursor(0, 0);
            lcd.print(palabrasClave[v]); // Muestra la palabra clave de la sección actual

            // Rellena el resto de la pantalla con espacios para que no quede basura visual

```



```

    for(int i = strlen(palabrasClave[v]); i < 16; i++) {
        lcd.print(' ');
    }
} else { // Acción B: Más de 1.2 segundos (Nuevo poema largo que avanza)
    unsigned long currentMillis = millis();

    // Controla la velocidad a la que avanza el poema nuevo sin usar delay
    if (currentMillis - lastScrollNuevoPoema > 300) {
        posNuevoPoema++;
        int largoPoemaNuevo = strlen(poemaNuevo);
        if (posNuevoPoema > largoPoemaNuevo - 16) {
            posNuevoPoema = 0; // Si llega al final, vuelve a empezar
        }
        lastScrollNuevoPoema = currentMillis;
    }

    lcd.setCursor(0, 0);
    for (int i = 0; i < 16; i++) {
        if (posNuevoPoema + i < strlen(poemaNuevo)) {
            lcd.print(poemaNuevo[posNuevoPoema + i]);
        } else {
            lcd.print(' ');
        }
    }
}
return; // Sale del loop para que no se ejecute el potenciómetro ni avance el verso normal
}
else if (b1 || b2) {
    // Si se presiona solo 1 botón (o se soltó uno estando en la Acción B)
    if (estadoActual != 1) {
        estadoActual = 1;
        lcd.clear(); // Limpia para asegurar que se borre el poemaNuevo si veníamos de ahí
    }

    // Imprime el texto original tal cual estaba, congelado en su posición actual
    int largoVerso = strlen(versosPoema[v]);
    lcd.setCursor(0, 0);
    for (int i = 0; i < 16; i++) {
        if (pos + i < largoVerso && pos + i >= 0) {
            lcd.print(versosPoema[v][pos + i]);
        } else {
            lcd.print(' ');
        }
    }
    return; // Sale del loop para que no actualice la posición y se quede congelado
}
else {
    // Ningún botón presionado: Funcionamiento normal con el potenciómetro
    if (estadoActual != 0) {
        estadoActual = 0;
        lcd.clear(); // Limpia la pantalla para quitar rastro de las acciones de botones
    }

    // ----- lectura pote -----
    // función para leer el potenciómetro
    // lectura de pin A0
    // conectar pin 2 de pote
    // lectura va de 0 a 1024
    poteLectura = analogRead(potePatita);

    // division de lectura de pote
    // valor resultante va de 0 a 255
    poteFiltrado = filtrarConDivision(poteLectura, 4);

    // imprimir en el monitor serial el poteFiltrado
    Serial.print("valor filtrado ");
    Serial.println(poteFiltrado);
    // ----- fin lectura pote -----

    if (poteFiltrado >= 135) {
        direccion = 1; // Avanzar
        // Mapea desde 135 (el mínimo para avanzar) hasta 255 (velocidad máxima)
        pausa = map(poteFiltrado, 135, 255, 600, 50);
    }
}

```

```

else if (poteFiltrado <= 120) {
  direccion = -1; // Retroceder
  // Mapea desde 120 (el mínimo para retroceder) hasta 0 (velocidad máxima en reversa)
  pausa = map(poteFiltrado, 120, 0, 600, 50);
}
else {
  direccion = 0; // Pausa / Zona muerta al centro (valores entre 121 y 134)
  pausa = 200; // Pequeño delay de espera
}

// calcula el verso actual
int totalVersos = sizeof(versosPoema) / sizeof(versosPoema[0]);
int largoVerso = strlen(versosPoema[v]);
int pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;

// imprimir el texto en la pantalla
lcd.setCursor(0, 0);
for (int i = 0; i < 16; i++) {
  if (pos + i < largoVerso && pos + i >= 0) {
    lcd.print(versosPoema[v][pos + i]);
  } else {
    lcd.print(' ');
  }
}

// 4. Aplicar la velocidad calculada
delay(pausa);

// 5. Actualizar la posición para el siguiente ciclo
pos += direccion;

// 6. Lógica para cambiar de verso si llegamos al límite (avanzando o retrocediendo)
if (pos > pasosTotales) {
  // Si avanza más allá del verso actual, pasa al siguiente
  pos = 0;
  v++;
  if (v >= totalVersos) v = 0; // Vuelve al inicio si terminó el poema
  lcd.clear();
  delay(100); // Pausa visual al cambiar de línea
}
else if (pos < 0) {
  // Si retrocede más allá del inicio, vuelve al verso anterior
  v--;
  if (v < 0) v = totalVersos - 1; // Va al último verso si retrocede desde el inicio

  // Recalcula el tamaño del nuevo verso para posicionarse al final de este
  largoVerso = strlen(versosPoema[v]);
  pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;
  pos = pasosTotales;

  lcd.clear();
  delay(100);
}
}
}

```

User prompt: El código funciona a la perfección, pero por temas de legibilidad y funcionamiento necesito que modifiques el código en la parte de la acción a: Ahora tenemos que: si los 2 botones se mantienen presionados menos de 1 segundo: En base a la sección del texto mostrado, se visualizará una palabra clave que nosotros elegimos por cada verso. Pero lo que ocurre con esta palabra es que aparece por tan poco tiempo que no se puede leer ya que desaparece al instante. Entonces necesito que me ayudes modificando el código para hacer que cada vez que aparezca cada una de estas palabras clave, duren exactamente 3 segundos en pantalla luego de realizar la acción a, es decir: apretar ambos botones a la vez por menos de un segundo. Nuevamente no modifiques nada más que no sea necesario en el código. Sigue manteniendo textos originales.

Response: ¡Claro, tiene todo el sentido! Lo que ocurría es que el Arduino es tan rápido que, en el milisegundo en que soltabas los botones, detectaba que ya no estaban presionados y borraba la pantalla inmediatamente para volver al poema normal.

Para arreglar esto de la forma más limpia y sin alterar tu lógica, le agregué una pequeña instrucción justo en el momento en que el código detecta que soltaste los botones. Si detecta que fue un toque corto (menos de 1.2 segundos), hace una pausa exacta de 3 segundos (`delay(3000)`) congelando la palabra en la pantalla antes de limpiar y continuar.

Aquí tienes el código ajustado. Solo modifiqué esos pequeños detalles en las transiciones de los botones y dejé intactos todos tus comentarios y el resto del funcionamiento:

```

#include <LiquidCrystal.h>

// versos del poema
const char *versosPoema[] = {
    "Cuando estan nuestras almas frente a frente,",
    "mudas, erguidas, fuertes, ya muy proximas,",
    "y sus alas se encienden al tocarse,",
    "en cada punta curva ?que mal amargo",
    "puede hacernos la tierra, que no debiéramos",
    "quedarnos aqui, contentos? Piensalo. Al subir más alto,",
    "los angeles nos oprimirian y aspirarian",
    "a dejar caer algun aureo orbe de canto perfecto",
    "en nuestro hondo, querido silencio. Quedemonos",
    "mejor en la tierra, Amado mio, donde los animos",
    "contrarios e injustos de los hombres retroceden",
    "y aislan a los espíritus puros, y permiten",
    "un lugar donde estar y amar por un día,",
    "con la oscuridad y la hora de la muerte rodeandolo."
};

// corresponde a los pines que utiliza la pantalla
// pantalla lcd verde 16 x 02 con controlador SPLC780D1 o HD44780
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// texto que se muestra al inciar el dispositivo
const char textoInicialCC[] = "[PLACEHOLDER] - CC BY-SA 4.0"; // licencia de uso, Creative Commons BY-SA 4.0
const char textoInicialTitulo[] = "Cuando nuestras dos almas se eleven"; // titulo del poema
const char textoInicialAutora[] = "Elizabeth Barret Brown"; // autora del poema

// --- variables para los botones y nuevas acciones ---
const int boton1Pin = 6;
const int boton2Pin = 7;
int estadoActual = 0; // 0: Normal, 1: Congelado (1 botón), 2: Dos botones presionados
unsigned long tiempoInicioDosBotones = 0;
unsigned long lastScrollNuevoPoema = 0;
int posNuevoPoema = 0;

// palabras clave por cada verso
const char *palabrasClave[] = {
    "firme y fuerte", "acercandose", "estallan", "en cada", "tierra",
    "contentos", "oprimiran", "algun", "silencio", "amado mio",
    "injusto", "aislan", "lugar", "muerte"
};

// nuevo poema conformado por las palabras clave
const char poemaNuevo[] = "firme y fuerte acercandose estallan en cada tierra contentos oprimiran algun silencio amado m

//----- variables pote -----
// variables y constantes
// para lectura potenciometro
const int potePatita = A0;
int poteLectura = -1;
int poteFiltrado = -1;

// funcion entera
// para tomar una variable entera original
// y dividirla por otro entero para perder resolucion
int filtrarConDivision(int valor, int divisor) {
    int resultado = valor / divisor;
    return resultado;
}
// ---- fin variables pote ----

// determinar dirección y calcular velocidad
// relacionada al desplazamiento del poema
int direccion = 0; // variable asociada a si el texto avanza o retrocede
int pausa = 0;
int v = 0;
int pos = 0; // variable que determina la posición del texto

// ----- inicio de funcionamiento -----

```

```

void setup() {

  Serial.begin(9600);
  lcd.begin(16, 2); //define el tamaño de la pantalla

  // Configuración de pines para los botones usando resistencias externas (pull-down)
  pinMode(boton1Pin, INPUT);
  pinMode(boton2Pin, INPUT);

  // --- texto inicial 1: textoInicialCC / Creative Commons BY - SA ---

  lcd.setCursor(0, 0); //define la seccion superior de la pantalla
  for(int i = 0; i < 16 && textoInicialCC[i] != '\0'; i++) {
    lcd.print(textoInicialCC[i]);
  }
  lcd.setCursor(0, 1); //define la seccion inferior de la pantalla
  lcd.print(textoInicialCC + 16);

  delay(4000);
  lcd.clear();

  // --- texto inicial 2: Carrusel de textoInicialB en la fila inferior (0, 0) ---
  int largoB = strlen(textoInicialTitulo); // calculam el largo (35 letras)

  // calcula cuántos pasos debe avanzar para mostrarlo todo
  // si el texto es más corto de 16, no se mueve (0 pasos)
  int pasosTotales = (largoB > 16) ? (largoB - 16 + 3) : 0; // +3 para dejar unos espacios al final

  for(int pos = 0; pos <= pasosTotales; pos++) {
    lcd.setCursor(0, 0);

    // imprime la "ventana" de 16 caracteres
    for(int i = 0; i < 16; i++) {
      if (pos + i < largoB) {
        lcd.print(textoInicialTitulo[pos + i]);
      } else {
        lcd.print(' '); // rellena con espacios en blanco cuando se acaba el texto
      }
    }

    // si esta en el primer cuadro (pos = 0), hace una pausa más larga
    // para que se pueda empezar a leer antes de que se mueva
    if (pos == 0) {
      delay(2000);
    } else {
      delay(500); // velocidad del carrusel (350ms por letra)
    }
  }

  lcd.clear();

  // --- texto inicial 3: textoInicialC / autora---
  lcd.setCursor(0, 0);
  for(int i = 0; i < 16 && textoInicialAutora[i] != '\0'; i++) {
    lcd.print(textoInicialAutora[i]);
  }
  lcd.setCursor(0, 1);
  lcd.print(textoInicialAutora + 16);

  delay(4000);
  lcd.clear();
}

void loop() {

  // Lectura de los botones (HIGH significa presionado porque usamos configuración pull-down)
  bool b1 = digitalRead(boton1Pin) == HIGH;
  bool b2 = digitalRead(boton2Pin) == HIGH;

  // LÓGICA DE LOS BOTONES
  if (b1 && b2) {

```

```

// Si se presionan ambos botones
if (estadoActual != 2) {
    estadoActual = 2;
    tiempoInicioDosBotones = millis();
    lcd.clear(); // Borra el texto de golpe
    posNuevoPoema = 0; // Prepara el nuevo poema desde el inicio
    lastScrollNuevoPoema = millis();
}

unsigned long tiempoPresionado = millis() - tiempoInicioDosBotones;

if (tiempoPresionado < 1200) { // Acción A: Menos de 1.2 segundos (1200 ms)
    lcd.setCursor(0, 0);
    lcd.print(palabrasClave[v]); // Muestra la palabra clave de la sección actual

    // Rellena el resto de la pantalla con espacios para que no quede basura visual
    for(int i = strlen(palabrasClave[v]); i < 16; i++) {
        lcd.print(' ');
    }
} else { // Acción B: Más de 1.2 segundos (Nuevo poema largo que avanza)
    unsigned long currentMillis = millis();

    // Controla la velocidad a la que avanza el poema nuevo sin usar delay
    if (currentMillis - lastScrollNuevoPoema > 300) {
        posNuevoPoema++;
        int largoPoemaNuevo = strlen(poemaNuevo);
        if (posNuevoPoema > largoPoemaNuevo - 16) {
            posNuevoPoema = 0; // Si llega al final, vuelve a empezar
        }
        lastScrollNuevoPoema = currentMillis;
    }

    lcd.setCursor(0, 0);
    for (int i = 0; i < 16; i++) {
        if (posNuevoPoema + i < strlen(poemaNuevo)) {
            lcd.print(poemaNuevo[posNuevoPoema + i]);
        } else {
            lcd.print(' ');
        }
    }
}
return; // Sale del loop para que no se ejecute el potenciómetro ni avance el verso normal
}

else if (b1 || b2) {
    // Si se presiona solo 1 botón (o se soltó uno estando en la Acción B)

    // NUEVO: Si venimos de soltar los dos botones y fue un toque corto, pausamos 3 segundos
    if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) {
        delay(3000); // Mantiene la palabra clave en pantalla exactamente 3 segundos
    }

    if (estadoActual != 1) {
        estadoActual = 1;
        lcd.clear(); // Limpia para asegurar que se borre el poemaNuevo si veníamos de ahí
    }

    // Imprime el texto original tal cual estaba, congelado en su posición actual
    int largoVerso = strlen(versosPoema[v]);
    lcd.setCursor(0, 0);
    for (int i = 0; i < 16; i++) {
        if (pos + i < largoVerso && pos + i >= 0) {
            lcd.print(versosPoema[v][pos + i]);
        } else {
            lcd.print(' ');
        }
    }
    return; // Sale del loop para que no actualice la posición y se quede congelado
}

else {
    // Ningún botón presionado: Funcionamiento normal con el potenciómetro

    // NUEVO: Si venimos de soltar los dos botones y fue un toque corto, pausamos 3 segundos
    if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) {
        delay(3000); // Mantiene la palabra clave en pantalla exactamente 3 segundos
    }
}

```

```

}

if (estadoActual != 0) {
    estadoActual = 0;
    lcd.clear(); // Limpia la pantalla para quitar rastro de las acciones de botones
}

// ----- lectura pote -----
// función para leer el potenciómetro
// lectura de pin A0
// conectar pin 2 de pote
// lectura va de 0 a 1024
poteLectura = analogRead(potePatita);

// division de lectura de pote
// valor resultante va de 0 a 255
poteFiltrado = filtrarConDivision(poteLectura, 4);

// imprimir en el monitor serial el poteFiltrado
Serial.print("valor filtrado ");
Serial.println(poteFiltrado);
// ----- fin lectura pote -----

if (poteFiltrado >= 135) {
    direccion = 1; // Avanzar
    // Mapea desde 135 (el mínimo para avanzar) hasta 255 (velocidad máxima)
    pausa = map(poteFiltrado, 135, 255, 600, 50);
}
else if (poteFiltrado <= 120) {
    direccion = -1; // Retroceder
    // Mapea desde 120 (el mínimo para retroceder) hasta 0 (velocidad máxima en reversa)
    pausa = map(poteFiltrado, 120, 0, 600, 50);
}
else {
    direccion = 0; // Pausa / Zona muerta al centro (valores entre 121 y 134)
    pausa = 200; // Pequeño delay de espera
}

// calcula el verso actual
int totalVersos = sizeof(versosPoema) / sizeof(versosPoema[0]);
int largoVerso = strlen(versosPoema[v]);
int pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;

// imprimir el texto en la pantalla
lcd.setCursor(0, 0);
for (int i = 0; i < 16; i++) {
    if (pos + i < largoVerso && pos + i >= 0) {
        lcd.print(versosPoema[v][pos + i]);
    } else {
        lcd.print(' ');
    }
}

// 4. Aplicar la velocidad calculada
delay(pausa);

// 5. Actualizar la posición para el siguiente ciclo
pos += direccion;

// 6. Lógica para cambiar de verso si llegamos al límite (avanzando o retrocediendo)
if (pos > pasosTotales) {
    // Si avanza más allá del verso actual, pasa al siguiente
    pos = 0;
    v++;
    if (v >= totalVersos) v = 0; // Vuelve al inicio si terminó el poema
    lcd.clear();
    delay(100); // Pausa visual al cambiar de línea
}
else if (pos < 0) {
    // Si retrocede más allá del inicio, vuelve al verso anterior
    v--;
    if (v < 0) v = totalVersos - 1; // Va al último verso si retrocede desde el inicio

    // Recalcula el tamaño del nuevo verso para posicionarse al final de este

```

```

largoVerso = strlen(versosPoema[v]);
pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0;
pos = pasosTotales;

lcd.clear();
delay(100);
}
}
}

```

User prompt: Nuevamente el código funciona bien, sin embargo, quiero saber algo específicamente en el código: ¿cuanto es el tiempo que tengo que estar pulsando los dos botones para que aparezca poemaNuevo? y ¿dónde modifico ese tiempo específicamente en el código? #include <LiquidCrystal.h> // versos del poema const char *versosPoema[] = { "Cuando estan nuestras almas frente a frente," "mudas, erguidas, fuertes, ya muy proximas," "y sus alas se encienden al tocarse," "en cada punta curva ?que mal amargo", "puede hacemos la tierra, que no debiéramos", "quedarnos aqui, contentos? Piensalo. Al subir más alto," "los angeles nos oprimiran y aspirarian", "a dejar caer algun aureo orbe de canto perfecto", "en nuestro hondo, querido silencio. Quedemonos", "mejor en la tierra, Amado mio, donde los animos", "contrarios e injustos de los hombres retroceden", "y aislan a los espiritus puros, y permiten", "un lugar donde estar y amar por un dia," "con la oscuridad y la hora de la muerte rodeandolo." }; // corresponde a los pines que utiliza la pantalla // pantalla lcd verde 16 x 02 con controlador SPLC780D1 o HD44780 LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // texto que se muestra al iniciar el dispositivo const char textoInicialCC[] = "[PLACEHOLDER] - CC BY-SA 4.0"; // licencia de uso, Creative Commons BY-SA 4.0 const char textoInicialTitulo[] = "Cuando nuestras dos almas se eleven"; // titulo del poema const char textoInicialAutora[] = "Elizabeth Barret Brown"; // autora del poema // --- variables para los botones y nuevas acciones --- const int boton1Pin = 8; const int boton2Pin = 9; int estadoActual = 0; // 0: Normal, 1: Congelado (1 botón), 2: Dos botones presionados unsigned long tiempoInicioDosBotones = 0; unsigned long lastScrollNuevoPoema = 0; int posNuevoPoema = 0; // palabras clave por cada verso const char *palabrasClave[] = { "firme y fuerte", "acercandose", "estallan", "en cada", "tierra", "contentos", "oprimiran", "algun", "silencio", "amado mio", "injusto", "aislan", "lugar", "muerte" }; // nuevo poema conformado por las palabras clave const char poemaNuevo[] = "firme y fuerte acercandose estallan en cada tierra contentos oprimiran algun silencio amado mio injusto aislan lugar muerte"; //----- variables pote ----- // variables y constantes // para lectura potenciómetro const int potePatita = A0; int poteLectura = -1; int poteFiltrado = -1; // funcion entera // para tomar una variable entera original // y dividirla por otro entero para perder resolucioin int filtrarConDivision(int valor, int divisor) { int resultado = valor / divisor; return resultado; } // ---- fin variables pote ---- // determinar dirección y calcular velocidad // relacionada al desplazamiento del poema int direccion = 0; // variable asociada a si el texto avanza o retrocede int pausa = 0; int v = 0; int pos = 0; // variable que determina la posición del texto // ---- inicio de funcionamiento ---- void setup() { Serial.begin(9600); lcd.begin(16, 2); //define el tamaño de la pantalla // Configuración de pines para los botones usando resistencias externas (pull-down) pinMode(boton1Pin, INPUT); pinMode(boton2Pin, INPUT); // --- texto inicial 1: textoInicialCC / Creative Commons BY - SA --- lcd.setCursor(0, 0); //define la seccion superior de la pantalla for(int i = 0; i < 16 && textoInicialCC[i] != '\0'; i++) { lcd.print(textoInicialCC[i]); } lcd.setCursor(0, 1); //define la seccion inferior de la pantalla lcd.print(textoInicialCC + 16); delay(4000); lcd.clear(); // --- texto inicial 2: Carrusel de textoInicialB en la fila inferior (0, 0) --- int largoB = strlen(textoInicialTitulo); // calculam el largo (35 letras) // calcula cuántos pasos debe avanzar para mostrarlo todo // si el texto es más corto de 16, no se mueve (0 pasos) int pasosTotales = (largoB > 16) ? (largoB - 16 + 3) : 0; // para dejar unos espacios al final for(int pos = 0; pos <= pasosTotales; pos++) { lcd.setCursor(0, 0); // imprime la "ventana" de 16 caracteres for(int i = 0; i < 16; i++) { if (pos + i < largoB) { lcd.print(textoInicialTitulo[pos + i]); } else { lcd.print(" "); // rellena con espacios en blanco cuando se acaba el texto } } // si esta en el primer cuadro (pos = 0), hace una pausa más larga // para que se pueda empezar a leer antes de que se mueva if (pos == 0) { delay(2000); } else { delay(500); } // velocidad del carrusel (350ms por letra) } lcd.clear(); // --- texto inicial 3: textoInicialC / autora --- lcd.setCursor(0, 0); for(int i = 0; i < 16 && textoInicialAutora[i] != '\0'; i++) { lcd.print(textoInicialAutora[i]); } lcd.setCursor(0, 1); lcd.print(textoInicialAutora + 16); delay(4000); lcd.clear(); } void loop() { // Lectura de los botones (HIGH significa presionado porque usamos configuración pull-down) bool b1 = digitalRead(boton1Pin) == HIGH; bool b2 = digitalRead(boton2Pin) == HIGH; // LÓGICA DE LOS BOTONES if (b1 && b2) { // Si se presionan ambos botones if (estadoActual != 2) { estadoActual = 2; tiempoInicioDosBotones = millis(); lcd.clear(); // Borra el texto de golpe posNuevoPoema = 0; // Prepara el nuevo poema desde el inicio lastScrollNuevoPoema = millis(); unsigned long tiempoPresionado = millis() - tiempoInicioDosBotones; if (tiempoPresionado < 1000) { // Acción A: Menos de 1.2 segundos (1200 ms) lcd.setCursor(0, 1); lcd.print(palabrasClave[pos]); // Muestra la palabra clave de la sección actual // Rellena el resto de la pantalla con espacios para que no quede basura visual for(int i = strlen(palabrasClave[pos]); i < 16; i++) { lcd.print(" "); } } else { // Acción B: Más de 1.2 segundos (Nuevo poema largo que avanza) unsigned long currentMillis = millis(); // Controla la velocidad a la que avanza el poema nuevo sin usar delay if (currentMillis - lastScrollNuevoPoema > 300) { posNuevoPoema++; int largoPoemaNuevo = strlen(poemaNuevo); if (posNuevoPoema > largoPoemaNuevo - 16) { posNuevoPoema = 0; // Si llega al final, vuelve a empezar } lastScrollNuevoPoema = currentMillis; } lcd.setCursor(0, 1); for (int i = 0; i < 16; i++) { if (posNuevoPoema + i < strlen(poemaNuevo)) { lcd.print(poemaNuevo[posNuevoPoema + i]); } else { lcd.print(" "); } } return; // Sale del loop para que no se ejecute el potenciómetro ni avance el verso normal } else if (b1 || b2) { // Si se presiona solo 1 botón (o se soltó uno estando en la Acción B) // NUEVO: Si venimos de soltar los dos botones y fue un toque corto, pausamos 3 segundos if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) { delay(3000); // Mantiene la palabra clave en pantalla exactamente 3 segundos } if (estadoActual != 1) { estadoActual = 1; lcd.clear(); // Limpia para asegurar que se borre el poemaNuevo si veníamos de ahí } // Imprime el texto original tal cual estaba, congelado en su posición actual int largoVerso = strlen(versosPoema[v]); lcd.setCursor(0, 0); for (int i = 0; i < 16; i++) { if (pos + i < largoVerso && pos + i >= 0) { lcd.print(versosPoema[v][pos + i]); } else { lcd.print(" "); } } return; // Sale del loop para que no actualice la posición y se quede congelado } else { // Ningún botón presionado: Funcionamiento normal con el potenciómetro // Si venimos de soltar los dos botones y fue un toque corto if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) { delay(2000); // Mantiene la palabra clave en pantalla exactamente 2 segundos } if (estadoActual != 0) { estadoActual = 0; lcd.clear(); // Limpia la pantalla para quitar rastro de las acciones de botones } //----- lectura pote ----- // función para leer el potenciómetro // lectura de pin A0 // conectar pin 2 de pote // lectura va de 0 a 1024 poteLectura = analogRead(potePatita); // division de lectura de pote // valor resultante va de 0 a 255 poteFiltrado = filtrarConDivision(poteLectura, 4); // imprimir en el monitor serial el poteFiltrado Serial.print("valor filtrado "); Serial.println(poteFiltrado); // ----- fin lectura pote ----- if (poteFiltrado >= 135) { direccion = 1; // Avanzar // Mapea desde 135 (el mínimo para avanzar) hasta 255 (velocidad máxima) pausa = map(poteFiltrado, 135, 255, 600, 50); } else if (poteFiltrado <= 120) { direccion = -1; // Retroceder // Mapea desde 120 (el mínimo para retroceder) hasta 0 (velocidad máxima en reversa) pausa = map(poteFiltrado, 120, 0, 600, 50); } else { direccion = 0; // Pausa / Zona muerta al centro (valores entre 121 y 134) pausa = 200; // Pequeño delay de espera } // calcula el verso actual int totalVersos = sizeof(versosPoema) / sizeof(versosPoema[0]); int largoVerso = strlen(versosPoema[v]); int pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0; // imprimir el texto en la pantalla lcd.setCursor(0, 0); for (int i = 0; i < 16; i++) { if (pos + i < largoVerso && pos + i >= 0) { lcd.print(versosPoema[v][pos + i]); } else { lcd.print(" "); } } // 4. Aplicar la velocidad calculada delay(pausa); // 5. Actualizar la posición para el siguiente ciclo pos += direccion; // 6. Lógica para cambiar de verso si llegamos al límite (avanzando o retrocediendo) if (pos > pasosTotales) { // Si avanza más allá del verso actual, pasa al siguiente pos = 0; v++; if (v >= totalVersos) v = 0; // Vuelve al inicio si terminó el poema lcd.clear(); delay(100); // Pausa visual

```

al cambiar de línea } else if (pos < 0) { // Si retrocede más allá del inicio, vuelve al verso anterior v--; if (v < 0) v = totalVersos - 1; //
Va al último verso si retrocede desde el inicio // Recalcula el tamaño del nuevo verso para posicionarse al final de este largoVerso =
strlen(versosPoema[v]); pasosTotales = (largoVerso > 16) ? (largoVerso - 16 + 3) : 0; pos = pasosTotales; lcd.clear(); delay(100);
} }

```

Response: Para que aparezca el poemaNuevo (lo que llamamos la Acción B), tienes que mantener los dos botones presionados por **1 segundo** (es decir, 1000 milisegundos).

Como detalle, si te fijas en la línea donde está esa instrucción, el código dice 1000, pero el comentario que quedó al lado dice 1.2 segundos (1200 ms). El Arduino siempre le hará caso al número del código (1000), no al comentario.

Si quieres modificar este tiempo para que tarde más o menos en aparecer, tienes que cambiar el número en **tres lugares específicos** dentro del void loop() para que toda la lógica siga coordinada:

1. Donde el código decide si ejecuta la Acción A o la B: Busca esta línea: if (tiempoPresionado < 1000) { // Acción A: Menos de 1.2 segundos (1200 ms) (Aquí cambias el 1000 por el tiempo en milisegundos que prefieras. Por ejemplo, si quieres que sean 2 segundos exactos, pones 2000).

2. Donde el código revisa qué hacer si sueltas 1 solo botón: Un poco más abajo, busca esta línea: if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) { (Aquí debes cambiar el 1200 para que coincida exactamente con el número nuevo que pusiste en el paso 1).

3. Donde el código revisa qué hacer si sueltas los 2 botones: Casi en la última sección, busca esta línea idéntica a la anterior: if (estadoActual == 2 && (millis() - tiempoInicioDosBotones) < 1200) { (Igual que antes, cambia el 1200 por tu nuevo número).

Es súper importante que si cambias el tiempo, lo actualices en esos tres puntos; si no, el Arduino se puede confundir con los tiempos de pausa.

¿Te queda alguna otra duda sobre cómo ajustar esta parte o pasamos a ver algo más del funcionamiento?
